

Theory of Automata

CS-3005

Lecture 5

Mahzaib Younas

Lecturer Department of Computer Science

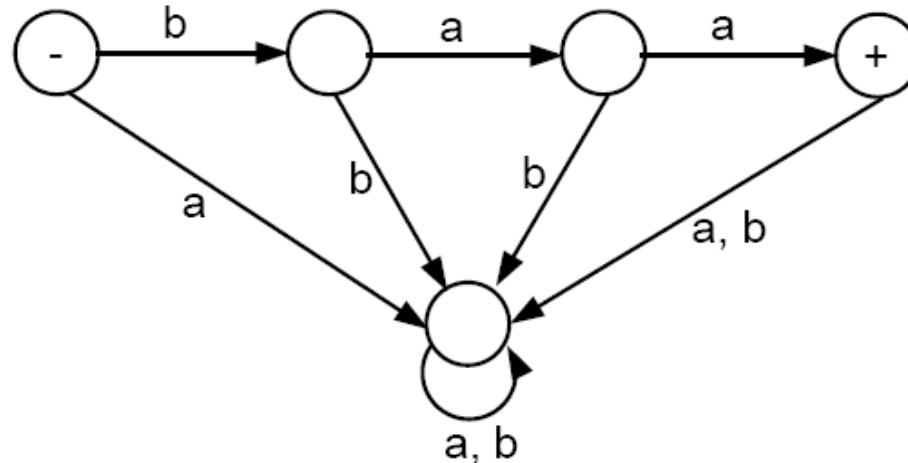
FAST NUCES CFD

Why we need Transition Graph?

- Relaxing the Restriction on Inputs
- Looking at TGs
- Generalized Transition Graphs
- Non-determinism
-

Relaxing the restrictions on inputs

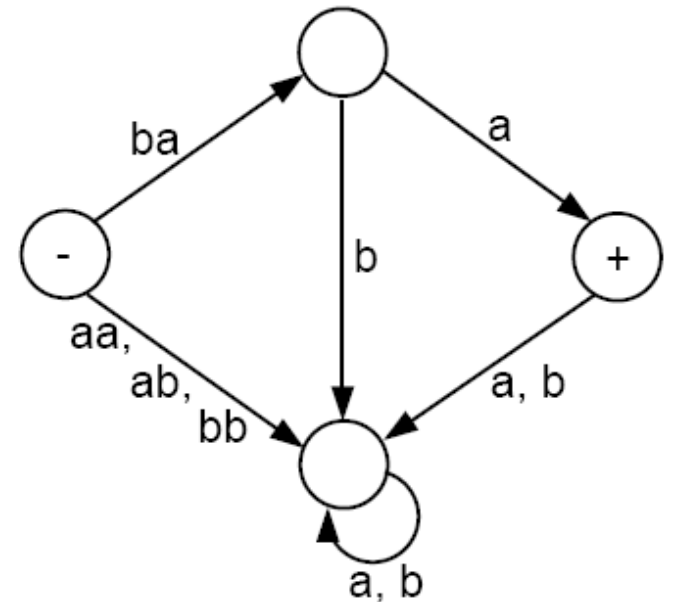
- Let's consider a very specialized FA that accepts **only the word baa**:



- Beginning at the start state, anything but the string baa will drop down into the garbage collecting state and never be seen again. **Even the string baabb** will fail.
- Hence the language accepted by this FA is $L = \{baa\}$

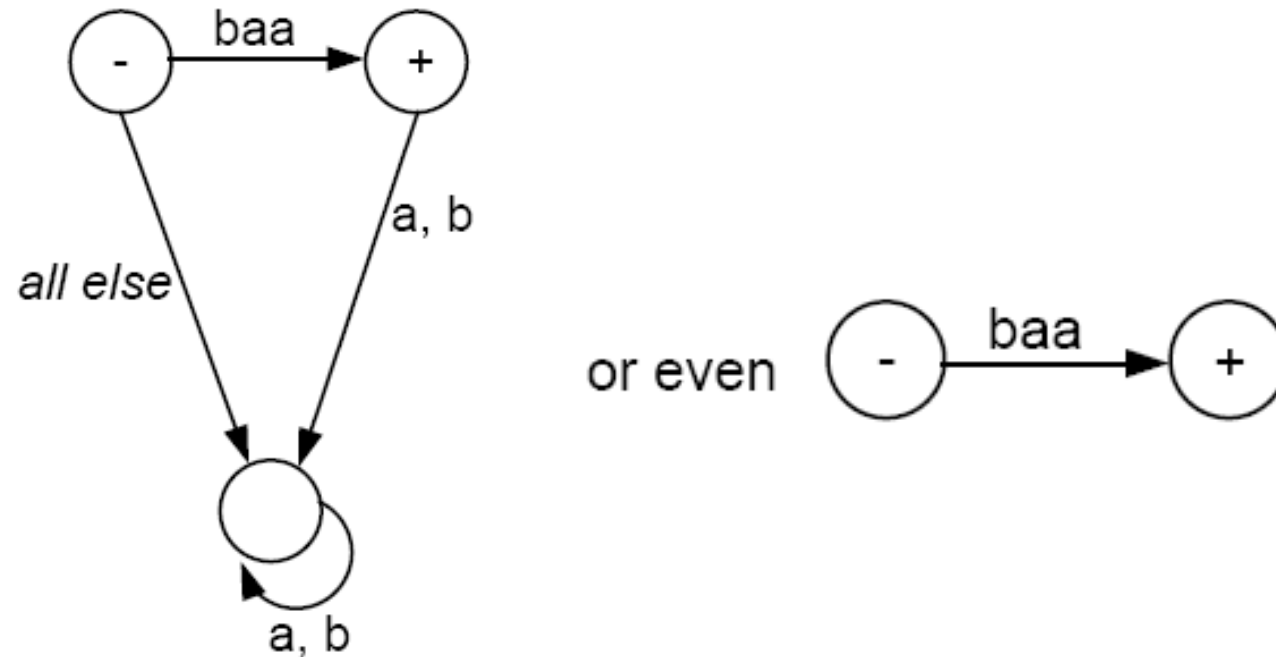
Relaxing the restrictions on inputs

- Let us relax the restriction that an FA can only read one letter from the input string at a time
- Suppose we now allow a machine to **read either one or two letters of the input string at a time**. Then we may design a machine that accepts only the word baa with fewer states as the one below:



Relaxing the restrictions on inputs

- If we go further to allow a machine to read up to three letters of the input string at a time, then we may design the machine accepting only the word baa with even fewer states as follows:



Relaxing the restrictions on inputs

- The picture on the last page tells us that when the input fails to be of the desired form, we must go to the garbage collection state and read through the rest of the input, knowing that we can never leave there.
- The picture on the last page introduces some problems:
 - – If we begin in the start (-) state and the first letter of the input is an a, we have no direction as to what to do.
 - – We also have problem even with the input baaa. The first three letters take us to the accept state, but then the picture does not tell us where to go when the last a is read. (Remember that according to the rules of FAs, one cannot stop reading input letters until the input string completely runs out.)
- We may assume, as a convention, that there is some garbage collection state associated with the picture on the right, but we do not draw it; and that we must go and stay there when the input string fails to be of the desired form.
- With this assumption, we can consider the two pictures above to be equivalent, in the sense that they accept the exact same language.

Relaxing the restrictions on inputs

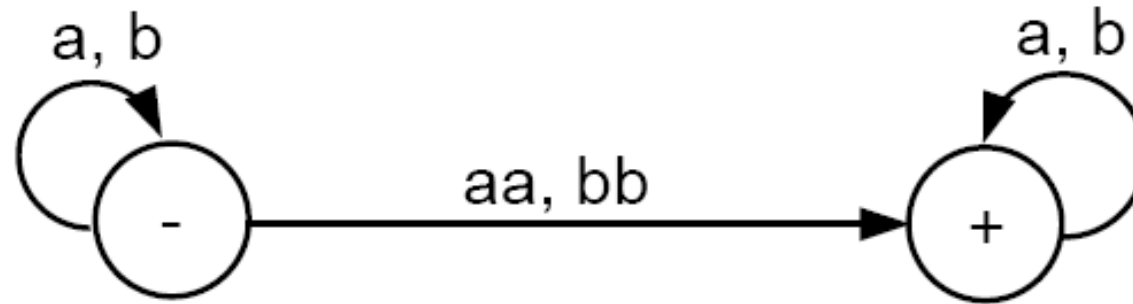
- Rather than trying to imagine a garbage collection state as described above, it is more standard to introduce a new term to describe what happens when an input is running on a machine and gets into a state from which it cannot escape, even though it has not yet been fully read.
- **Definition:** When an input string that **has NOT been completely read reaches a state (final or otherwise) that it cannot leave because there is no outgoing edge that it may follow**, we say that the input (or the machine) crashes at that state. **Execution then terminates and the input must be rejected.**

Relaxing the restrictions on inputs

- Note that on an FA it is not possible for any input to crash because there are always an outgoing a-edge and an outgoing b-edge from each state. As long as there remains input letters unread, progress is possible.
- There are now two different ways that an input can be rejected:
 - It could peacefully trace a path ending in a non-final state,
 - it could crash while being processed.

Relaxing the restrictions on inputs

- If we hypothesize that a machine can read one or two letters at a time, then one may build a machine using only two states that can recognize all words containing a double letter (aa or bb) as follows:



- We must now realize that we have changed something more fundamental than just the way the edges are labeled or the number of letters read at a time: This machine makes us exercise some choice, i.e., we must decide how many letters to read from the input string each time.

Definition of Transition Graph

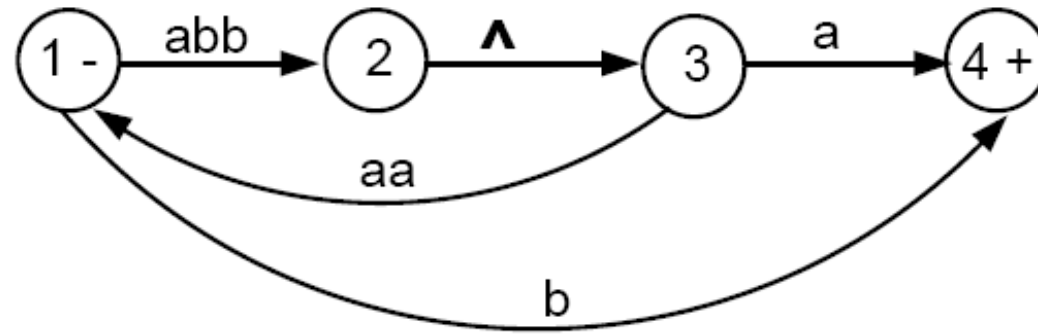
- A transition graph, abbreviated TG, is a collection of three things:
 - 1. A finite set of states, at least one of which is designated as the start state (-), and some (maybe none) of which are designated as final states (+).
 - 2. An alphabet Σ of possible input letters from which input strings are formed.
 - 3. A finite set of transitions (edge labels) that show how to go from some states to some others, based on reading specified substrings of input letters (possibly even the null string Λ).

Definition of Transition Graph

- We should note the following from the definition of a TG:
 1. Clause 3 in the definition means that every edge is labeled by **some string or strings of letters**, not necessarily only one letter.
 2. *We are NOT requiring that there be any specific number of edges emanating from any state:* Some states may have no edge coming out at all, and some may have thousands (e.g., edges labeled a, aa, aaa, aaaa, ...).
 3. A **successful path** through a TG is a series of edges forming a path beginning at **some start state** (there may be several) and ending at a final state.
 4. If we concatenate in order the strings of letters that label each edge in a successful path, **we produce a word that is accepted by this TG.**

Example

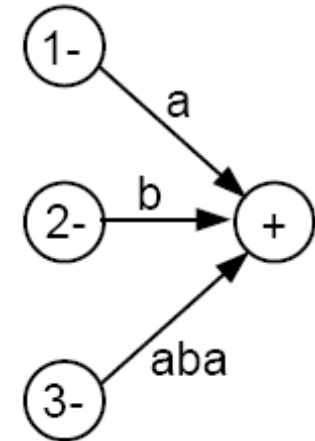
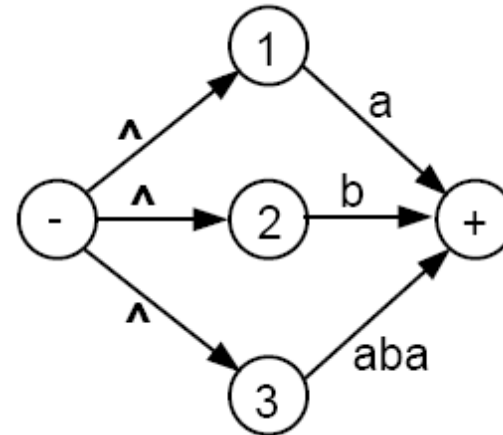
- For example, consider the following TG:



- The path from state 1 to state 2 to state 3 back to state 1 and then to the final state 4 corresponds to the string $(abb)(\Lambda)(aa)(b) = abbaab$.
- Some other accepted words are $abba$, $abbbaabba$, and b .
- When an edge is labeled with Λ , it means that we can take the ride it offers for free (without consuming any letter from the input string).

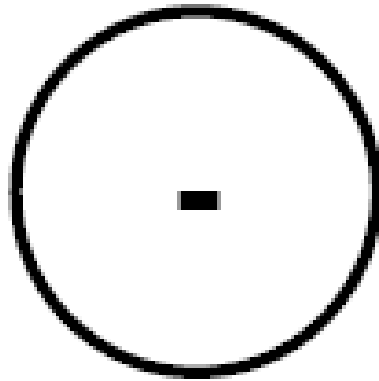
Example 2:

- Because we allow some edges to be traversed for free, it is logical to allow for the possibility of more than one start state, as illustrated below:



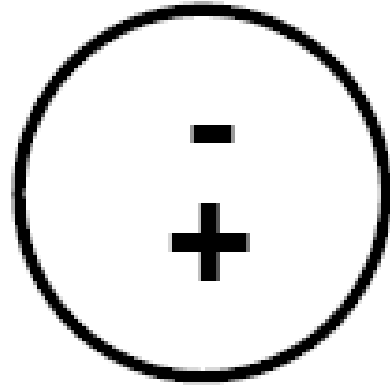
- These two machines are equivalent, in the sense that all the strings accepted by the first are accepted by the second and vice versa.
- Important Note: Every FA is also a TG. However, NOT every TG satisfies the definition of an FA.

Looking at TGs



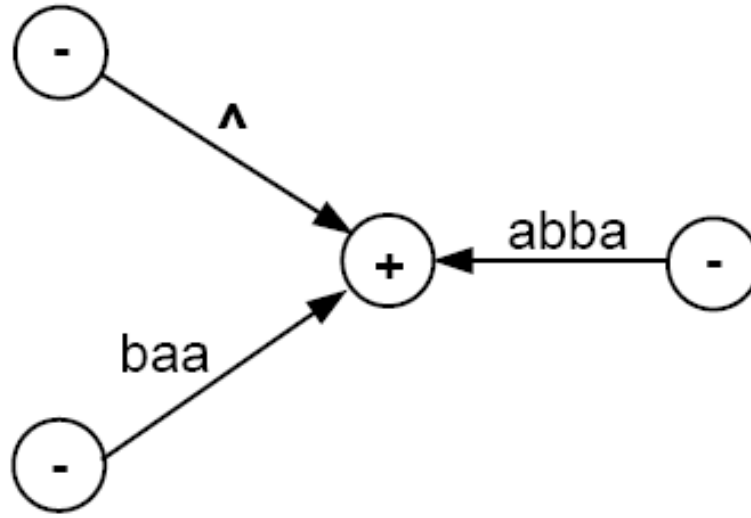
- This TG accepts nothing, not even the null string.
- To be able to accept anything, it must have a final state.

Null String



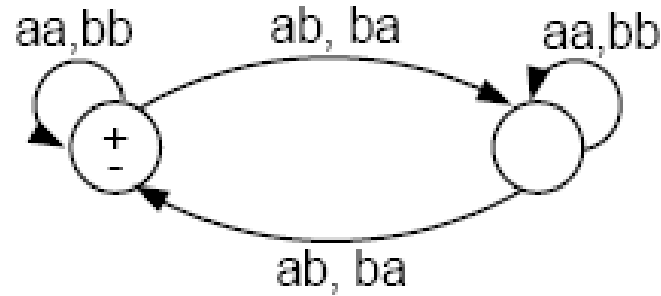
- This TG accepts only the null string Λ .
- Any other string cannot have a successful path to the final state through labels of edges because there are no edges (and hence no labels).
- Any TG in which some start state is also a final state will always accept the null string Λ . This is also true of FAs.

Example of String



- This machine accepts only the words Λ , baa, and abba.
- Anything read while in the final state will cause a crash, because the final state has no outgoing edges.

Example



- In this TG, every edge is labeled with a pair of letters. Thus, for a string to be accepted, it must have an even number of letters.
- Let's call the left state the **balanced state**, and the right state the **unbalanced state**.
- If the first pair of letters that we read is a double (aa or bb), then we stay in the balanced state. While in the balanced state, we have read an even number of a's and an even number of b's.

EVEN EVEN

- When a pair of unmatched letters is read (ab or ba), the machine flips over to the unbalanced state, meaning that it has read an odd number of a's and an odd number of b's.
- We do not return to the balanced state until another unmatched pair is read. The discovery of two unmatched pairs makes the total number of a's and the total number of b's read from the input string even again.
- This TG accepts exactly the language EVEN - EVEN.
- Recall that EVEN - EVEN is the language of all words containing an even number of a's and an even number of b's, including the null string Λ .

Basic Question

- How many start states a TG can have?
- .
- What is a successful path in TG?
- What is the language of a TG?

Basic Question

- How many start states a TG can have?

At least ONE.

- What is a successful path in TG?

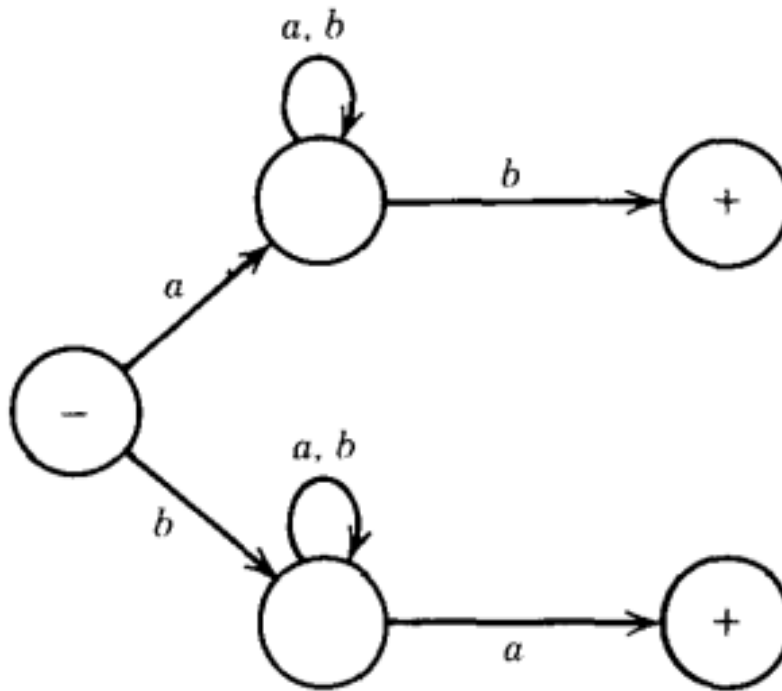
A **successful path** through a TG is a series of edges forming a path beginning at some start state (there may be several) and ending at a final state.

- What is the language of a TG?

If we concatenate in order the strings of letters that label each edge in a successful path, we produce a word that is accepted by this TG.

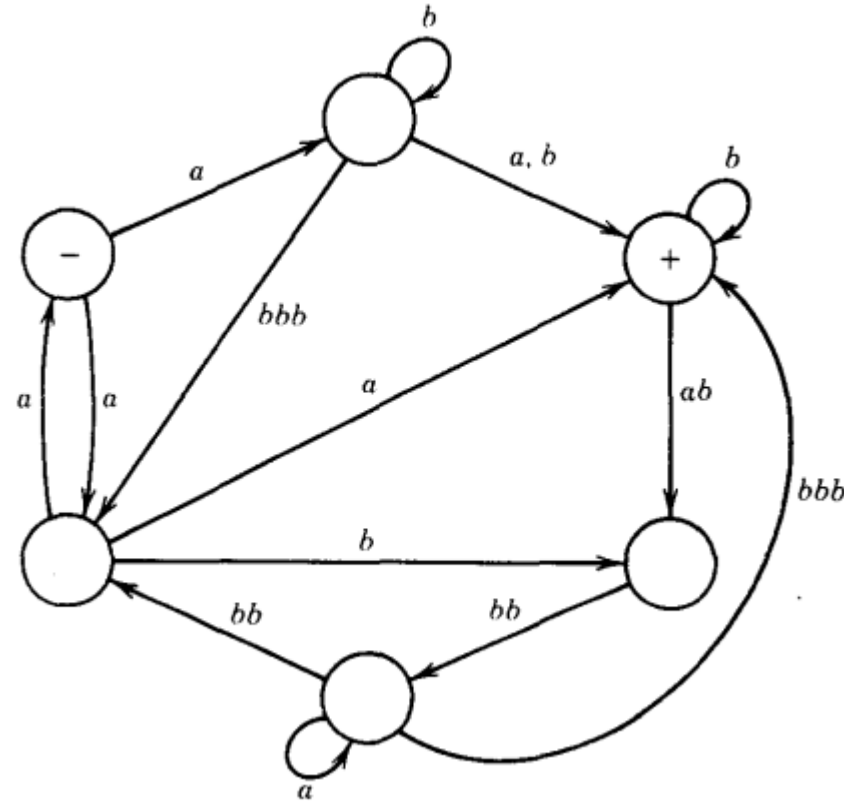
Practice Question:

- The following TG accepts the language of all words that begin and end with different letters.



Practice Question:

- Consider this TG Is the word `abbbabbbabba` accepted by this machine?
(Yes, in two ways.)



Generalized Transition Graphs (GTG)

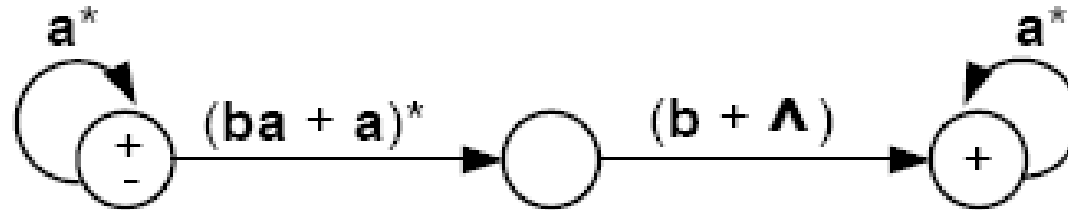
Generalized Transition Graphs (GTG)

A **generalized transition graph (GTG)** is a collection of three things:

1. A finite set of states, of which at least one is a start state and some (maybe none) are final states.
2. An alphabet Σ of input letters.
3. Directed edges connecting some pairs of states, each labeled with a regular expression.

Example:

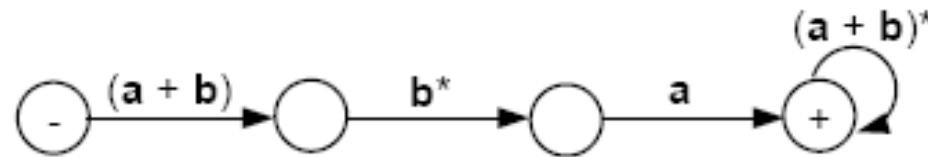
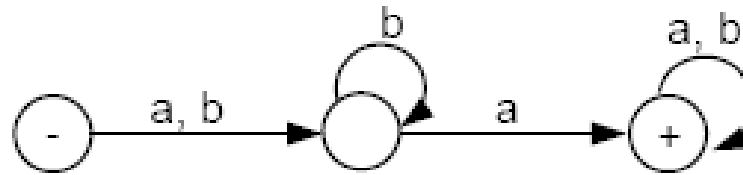
- Consider this GTG:



- This GTG accepts all strings without a double b.
- Note that the word containing the single letter b can take the free ride along the Λ -edge from start to middle, and then have letter b read to go to the final state.
- Typo in textbook: The first edge should be labeled $(ba + a)^*$ as in the figure above, not $(ab + a)^*$.

Example:

- Note that there is no difference between the Kleene star closure for regular expressions and a loop in transition graphs, as illustrated in the following figure:

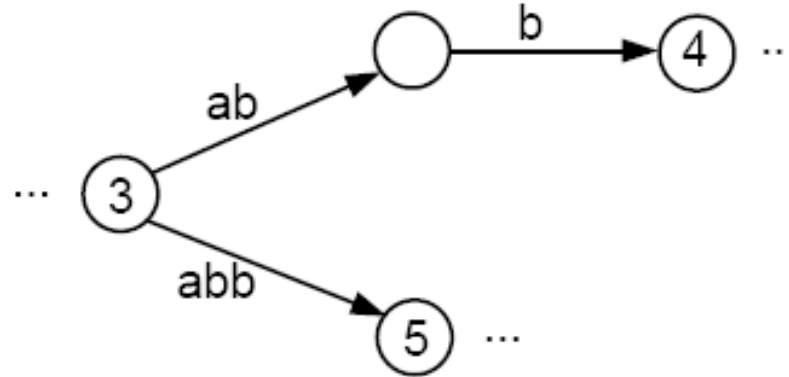


- In the first picture we may loop in the middle state or go to the third state. For not to loop, corresponds to taking the Λ choice from the b^* -edge in the second picture.

Non-Determinism

Non-Determinism

- We have already seen that in a TG, a particular string of input letters may trace through the machine on different paths, depending on our choice of grouping.



- This figure shows part of some TG.
- The input string *abb* can go from state 3 to state 4, or to state 5, depending on whether we read the letters two and one, or all three at once.

Non-Determinism

- The ultimate path through the machine is NOT determined by the input alone. Human choice becomes a factor in selecting the path. The machine does not make all its own determination.
- Therefore, we say that TGs are **nondeterministic**.

Comparison of FA, NFA, and TG

	FA	NFA	TG
Start State	one	one	One or more
Final State	Some or none	Some or none	Some or none
Edge Labels	Letters from Σ	Letter from Σ and lamda	Word from Σ^*
Number of edges from each state	One for each letter in Σ	Arbitrary	Arbitrary
Deterministic	Yes	No	No
Output	No	No	No